

CS1010J Lecture #10

Characters & Strings

Text processing



Department of Computer Science
School of Computing

OOP Program Structure Illustration

User (Driver) Class

- **main()** method
- other *static* helper methods if needed

Predefined Service Classes

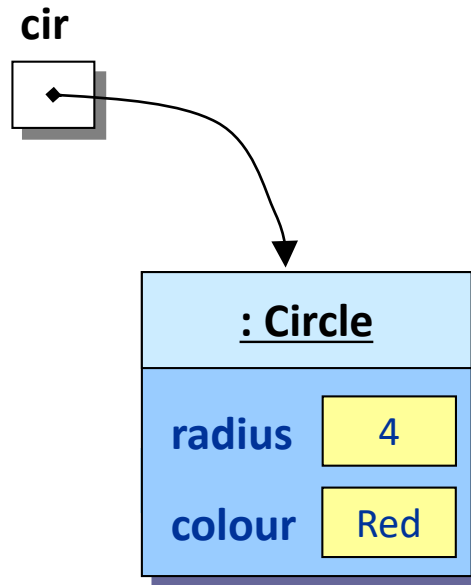
- Java library classes

Own Service Classes

- attributes
- constructors
- member methods

Object and Object Reference

```
Circle cir = new Circle(4, "Red");
```



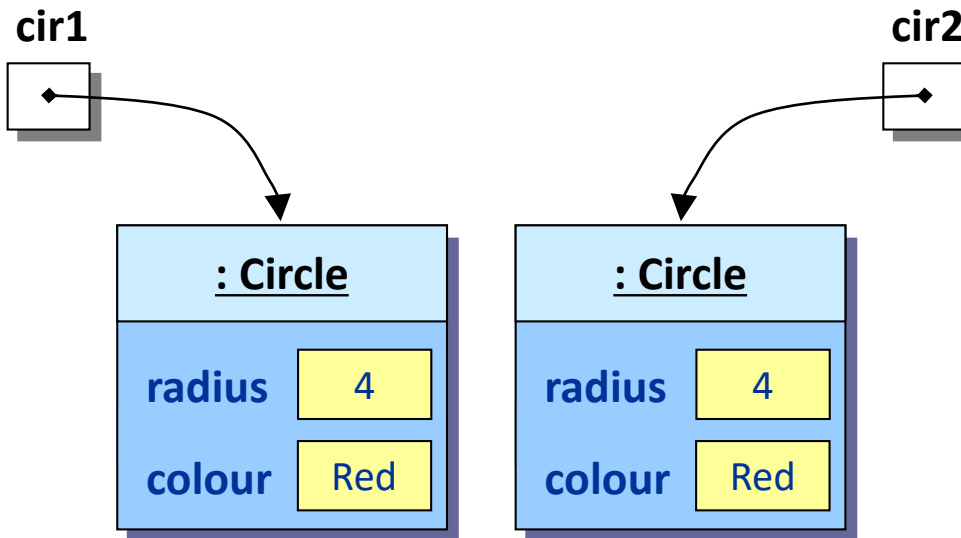
1. declare an object
reference `cir`

2. create a `Circle` object
and assign its memory
address to `cir`

- Objects are called *reference data types*.

Comparing Objects (1/2)

```
Circle cir1 = new Circle(4);  
Circle cir2 = new Circle(4);  
if (cir1 == cir2) {  
    System.out.println("They are identical");  
} else {  
    System.out.println("They are different");  
}
```



`if (cir1 == cir2)`
compares the **memory addresses** of the two Circle objects.

Comparing Objects (2/2)

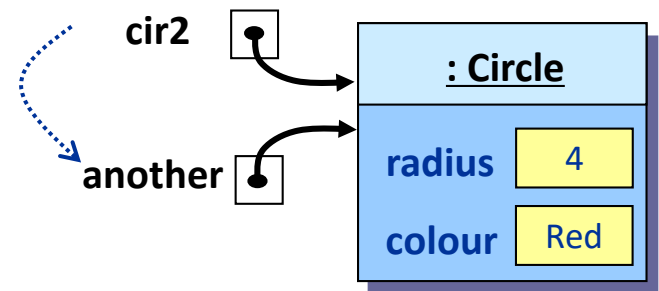
- We need to write our own method to compare the attributes of two objects.
 - By convention, such a method is named `equals()`.

```
class Circle {  
  
    // Other code omitted for brevity  
  
    // Compare the data of two circle objects  
    public boolean equals(Circle another) {  
        return this.radius == another.radius &&  
            this.colour.equals(another.colour);  
    }  
}
```

Object as Parameter

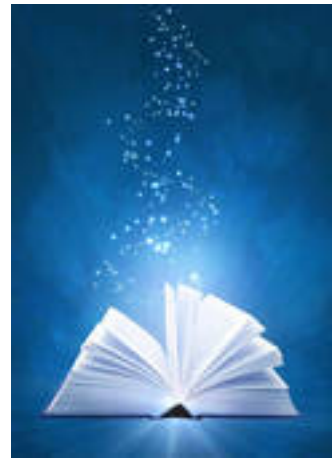
- All parameters in Java are **passed by value**.
 - **Value** of actual parameter is copied to the formal parameter

```
class TestCircle {  
    public static void main( String[] args ) {  
        if ( cir1.equals(cir2) ) {  
            System.out.println("Two circles are identical");  
        }  
        // Other code ...  
    }  
}  
  
class Circle {  
    // Other code ...  
    public boolean equals(Circle another) {  
        return this.radius == another.radius &&  
            this.colour.equals(another.colour);  
    }  
}
```



Learning Objectives

- At the end of this lecture, you should understand:
 - How to declare and manipulate data of the `char` data type.
 - The fundamental operations on `strings`.



Characters

- In Java, characters are represented using the data type **char**.
- Characters are written as symbols enclosed in single quotes.
 - Examples: `'G'`, `'8'`, `'*'`, `' '`, `'\n'`
- Characters are stored in computer memory using some form of encoding.
 - **ASCII** (American Standard Code for Information Interchange) is one of the document coding schemes widely used today.
 - Java uses **Unicode**, which includes ASCII, for representing char constants.

ASCII Encoding

	0	1	2	3	4	5	6	7	8	9
0	nul	soh	stx	etx	eot	enq	ack	bel	bs	ht
10	lf	vt	ff	cr	so	si	dle	dcl	dc2	dc3
20	cd4	nak	syn	etb	can	em	sub	esc	fs	gs
30	rs	us	sp	!	"	#	\$	%	&	'
40	(	)	*	+	,	-	.	/	0	1
50	2	3	4	5	6	7	8	9	:	;
60	<	=	>	?	@	A	B	C	D	E
70	F	G	H	I	J	K	L	M	N	O
80	P	Q	R	S	T	U	V	W	X	Y
90	Z	[	\	]	^	_	`	a	b	c
100	d	e	f	g	h	i	j	k	l	m
110	n	o	p	q	r	s	t	u	v	w
120	x	y	z	{	}		~	del		

For example,
character 'O' is 79
(row value 70 +
col value 9 = 79).

Demo #1 : Using Characters

```
class DemoCharacter {  
  
    public static void main(String[] args) {  
  
        char ch = 'a';  
        System.out.println( ch );  
        System.out.println( (int)ch );  
  
        int i = 65;  
        System.out.println( i );  
        System.out.println( (char)i );  
  
        if ( 'A' < ch ) { // compare ASCII values  
            System.out.println( "'A' is less than 'a'" );  
        }  
  
        System.out.println( 'd' - ch );  
    }  
}
```

print out a
character

print out ASCII value
of a character

```
a  
97  
65  
A  
'A' is less than 'a'  
3
```

Java Class : Characters

- The **Character** class defined in the `java.lang` package contains useful methods for processing characters, some of which are listed below.

Methods	Explanation
<code>isDigit(ch)</code>	Returns <code>true</code> if <code>ch</code> is a digit character, or <code>false</code> otherwise.
<code>isLetter(ch)</code>	Returns <code>true</code> if <code>ch</code> is a alphabet letter, or <code>false</code> otherwise.
<code>isWhitespace(ch)</code>	Returns <code>true</code> if <code>ch</code> is a whitespace character (e.g. blank, newline), or <code>false</code> otherwise.

- Example use:

```
boolean result = Character.isDigit('5');
```

*Note the syntax to
use class method*

Strings

- Conceptually, a string is a sequence of characters.
- In Java, **String** is a standard class defined in the **java.lang** package.
 - All string literals in Java programs, such as "**abc**", are implemented as instances of this class.
- A string can be concatenated with another, or even with numeric values using the **'+'** operator.
- Example:

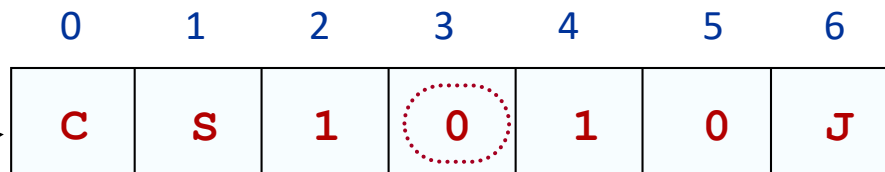
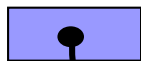
```
String claim = "I am rich";  
System.out.println(claim + " in knowledge");
```

Accessing Individual Elements

- Individual characters in a string is accessed with the `charAt()` method.
- Example:

```
String module = "CS1010J";  
System.out.println( module.charAt(3) );
```

module



Return the character
at position #3.

Inputting Strings

- **Scanner** class provides two methods to read strings:
 - ❑ To read a single word (i.e. read up to and exclude a white space) as a string, use the `next()` method.
 - ❑ To read a whole line (i.e. read up to <Enter>) as a string, use the `nextLine()` method.

```
Scanner sc = new Scanner(System.in);  
  
// Compare nextLine() and next()  
  
System.out.print("Enter name: ");  
String name1 = sc.nextLine();  
System.out.println("Name entered is " + name1);  
  
System.out.print("Enter name: ");  
String name2 = sc.next();  
System.out.println("Name entered is " + name2);
```

```
Enter name: Wilson Wee  
Name entered is Wilson Wee  
Enter name: Wilson Wee  
Name entered is Wilson
```

Demo #2 : Counting Vowels (1 / 2)



- Write a program `CountVowels.java` to count the number of vowels in a given string.
- Sample run:

```
Enter a string: WHAT is your name?  
Number of vowels: 6
```

Demo #2 : Counting Vowels (2/2)



```
import java.util.*;

class CountVowel {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String str = sc.nextLine().toUpperCase();

        int vowelCount = 0;
        for (int i = 0; i < str.length(); i++) {
            switch ( str.charAt(i) ) {
                case 'A': case 'E': case 'I': case 'O': case 'U':
                    vowelCount++;
            }
        }
        System.out.println("Number of vowels: " + vowelCount);
    }
}
```

*Returns the number of
characters in the string*

String Class : `length()` method

- To obtain the length of a **String** object, we use the `length()` method.
- Comparatively, to obtain the length (size) of an array, we use the `length` attribute.
- Example:

```
int[] arr = { 1, 2, 3, 4 };  
String str = "1234";
```

Length of array: 4
Length of string: 4

```
// Compare array length and string length()  
System.out.println("Length of array: " + arr.length );  
System.out.println("Length of string: " + str.length() );
```

Creating Strings

- In Java, strings are objects.
 - You may create a **String** object using the **new** operator.
 - But more often, we use a shortcut to assign a string literal directly to a **String** variable.
- Example:

```
String str1, str2;
```

```
str1 = new String("Java");
```

Use a string literal to construct a new string object

```
str2 = "I love CS1010J";
```

Assign a string literal directly

- In fact, the **String** class provides 15 constructors to create strings in different ways.

Comparing Strings

- With primitive data types, we have only one way to compare them: `<`, `<=`, `>`, `>=`, `==`
- Strings are objects. As we know, to compare objects,
 - We use `==` to check whether two object references point to the same object (less often used in practice).
 - We use `equals()` to check whether two objects have the same content.

```
Scanner sc = new Scanner(System.in);  
String str1 = sc.nextLine();  
String str2 = sc.nextLine();  
System.out.println(str1 == str2);  
System.out.println(str1.equals(str2));
```

```
Hello world!  
Hello world!  
false  
true
```

Demo #3 : Counting Java (1/2)

- Write a program `CountJava.java` to read a list of words and count how many times the word “Java” occurs in the input, ignoring the case. User input will stop when the word “STOP” is entered.
- Sample run:

```
Next word: Java
Next word: JAVA
Next word: JAva
Next word: JavA
Next word: Jaaaaava
Next word: STOP
"Java" appeared 4 times, ignoring case
```


Useful String Methods

- The String class provides a rich set of useful methods, some of which are listed below.
 - Please read the online Java API documentation for details.

Methods	Explanation
length	Returns the length of this string (i.e. number of characters contained).
toUpperCase	Converts all letters in this string to uppercase.
indexOf	Searches a given character (or string) in this string and returns the index of its first occurrence.
substring	Returns a substring of this string.
trim	Removes the leading and trailing spaces from this string.
split	Splits this string into an array of substrings.
equals	Returns true if this string has the same content as another.

An Array of Strings

■ Example:

```
// Given an integer representing a month (1 - 12),  
// return the name of that month  
public static String returnMonth(int index) {  
  
    String[] months = { "Jan", "Feb", "Mar", "Apr",  
                        "May", "Jun", "Jul", "Aug",  
                        "Sep", "Oct", "Nov", "Dec" };  
  
    return months[index - 1];  
}
```

- In fact, the main() method takes a parameter which is an array of String objects.
 - We may use it for command-line arguments (not covered).

```
public static void main(String[] args) {
```

The String Class is Immutable

- In Java, a String object is **immutable**.
- This means once a **String** object is created, it cannot be changed.
- The String methods we have discussed **do not** change the original string. Some of them actually create a new string from the original.
 - For example, **toUpperCase()** method creates and returns a new string from a given string in which all letters are in uppercase.

Today's Summary

Searching and Sorting

Characters

- ASCII values
- **Character** class and methods

Strings

- String declaration
- String input
- String comparison
- **String** class and methods

